

Name:Darshit kacha

Enrolment no:92400133089

Batch-A

Class:5ek1

Open ended problem -2

I am using python 3.14 so instead of audioop I am using librosa so according to that I am attaching here code .

Code :

```
import librosa import soundfile as
```

```
sf import numpy as np import
```

```
matplotlib.pyplot as plt from
```

```
scipy.signal import fftconvolve
```

```
# Load MP3 file samples,
```

```
sample_rate = librosa.load(
```

```
    "C:/Users/hp/Downloads/sare-jahan-se-acha-beautifully-remastered-30952.mp3",
```

```
sr=None,  mono=True
```

```
)
```

```
# Define convolution kernel kernel = np.array([1,
```

```
0, 1, 0, 1], dtype=np.float32)
```

```
# Perform convolution convoluted =
```

```
fftconvolve(samples, kernel, mode='same')
```

```
# Normalize
```

```
max_val = np.max(np.abs(convoluted)) if
```

```
max_val != 0:
```

```
convoluted = convoluted / max_val
```

```
# Save as WAV sf.write("output_convoluted.wav",  
convoluted, sample_rate)
```

```
print("Convoluted audio saved as 'output_convoluted.wav'.")
```

```
# Plot first 5000 samples plt.figure(figsize=(12,6))
```

```
plt.subplot(2,1,1)
```

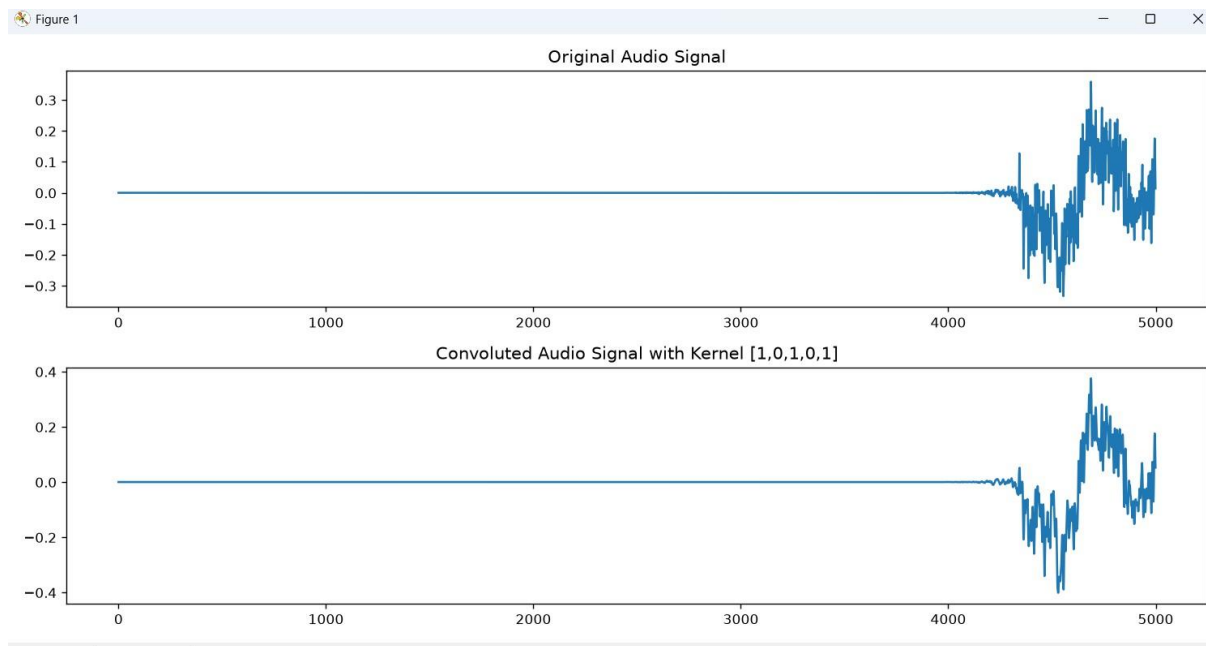
```
plt.plot(samples[:5000]) plt.title("Original  
Audio Signal")
```

```
plt.subplot(2,1,2) plt.plot(convoluted[:5000])
```

```
plt.title("Convoluted Audio Signal with Kernel [1,0,1,0,1]")
```

```
plt.tight_layout()
```

```
plt.show() output:
```



code :

```
import librosa import soundfile as
```

```
sf import numpy as np import
```

```
matplotlib.pyplot as plt from
```

```
scipy.signal import fftconvolve
```

```
audio_path = "C:/Users/hp/Downloads/sare-jahan-se-acha-beautifully-remastered30952.mp3"
```

```
samples, sample_rate = librosa.load(audio_path, sr=None, mono=True)
```

```
samples = samples / np.max(np.abs(samples))
```

```
impulse_responses = {
```

```
    "IR1": np.array([1, 0, 1, 0, 1], dtype=np.float32),
```

```
    "IR2": np.array([0.2, 0.5, 1, 0.5, 0.2], dtype=np.float32),
```

```
    "IR3": np.array([1, -0.5, 0.25], dtype=np.float32)
```

```
}
```

```
plt.figure(figsize=(12,8))
```

```
for index, (name, ir) in enumerate(impulse_responses.items()):
```

```
    convoluted = fftconvolve(samples, ir, mode='same')
```

```
    convoluted /= np.max(np.abs(convoluted))
```

```
    # Save Convoluted Audio
```

```
    sf.write(f"{name}_convoluted.wav", convoluted, sample_rate)
```

```
    ir_fft = np.fft.fft(ir, len(convoluted))
```

```
    conv_fft = np.fft.fft(convoluted)
```

```
    epsilon = 1e-6
```

```
    inverse_fft = conv_fft / (ir_fft + epsilon)
```

```
    restored = np.real(np.fft.ifft(inverse_fft))
```

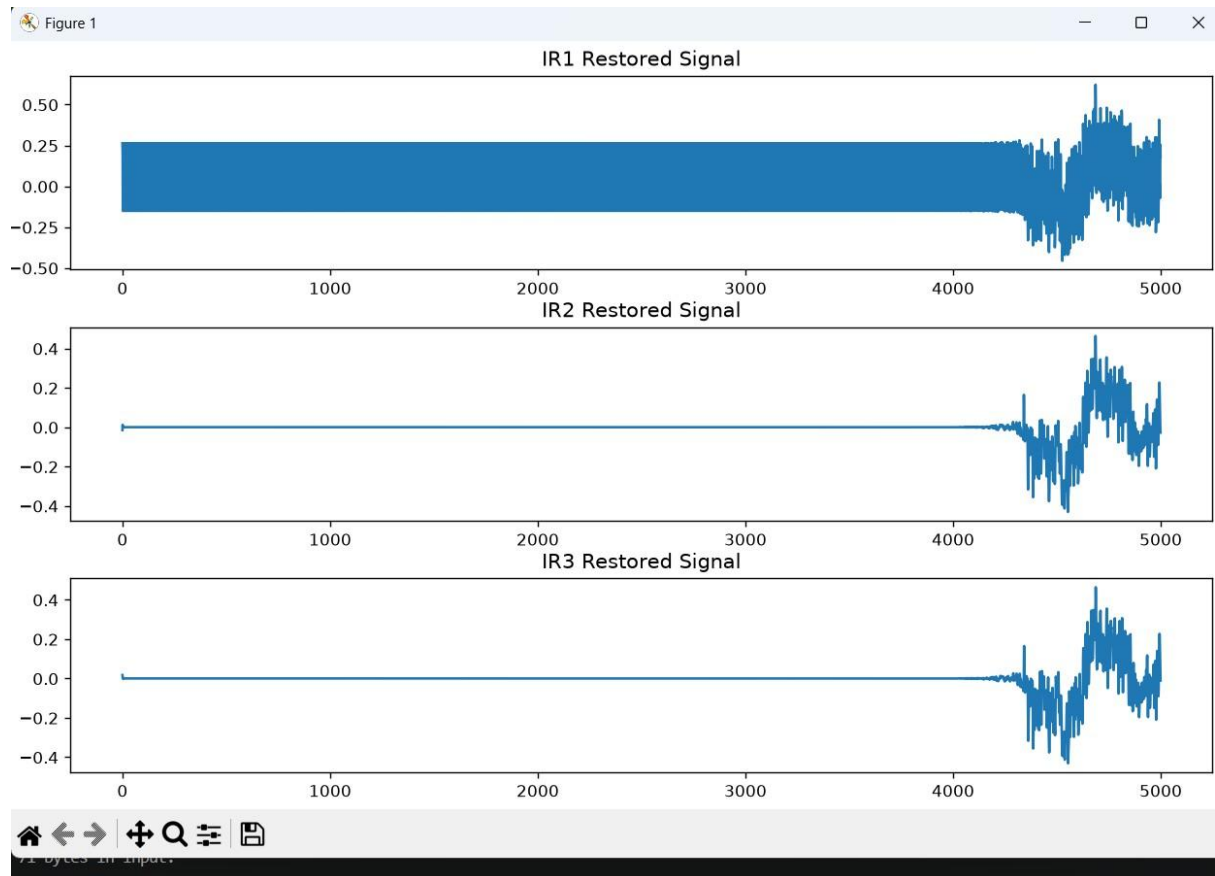
```
    restored /= np.max(np.abs(restored))
```

```
    sf.write(f"{name}_restored.wav", restored, sample_rate)    plt.subplot(3,1,index+1)
```

```
plt.plot(restored[:5000])    plt.title(f"{name} Restored Signal")
```

```
plt.tight_layout()
```

plt.show() output:



conclusion:

Some filters preserved the voice better, while others introduced more distortion or noise. Inverse filtering helped recover the original signal to some extent but could not completely remove the background music.

GitHub Link:

<https://github.com/Darshit1009/DSIP>